

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №4

Выполнил:
Студент группы ИУ5-35Б
Корольков И. А.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Листинг shop.py

```
from abc import ABC, abstractmethod

# ===== Strategy (поведенческий паттерн) =====
class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount: float) -> str:
        pass

class CardPayment(PaymentStrategy):
    def pay(self, amount: float) -> str:
        return f"Paid {amount} by card"

class PayPalPayment(PaymentStrategy):
    def pay(self, amount: float) -> str:
        return f"Paid {amount} via PayPal"

# ===== Factory Method (порождающий паттерн) =====
class PaymentFactory:
    @staticmethod
    def create_payment(method: str) -> PaymentStrategy:
        if method == "card":
            return CardPayment()
        if method == "paypal":
            return PayPalPayment()
        raise ValueError("Unknown payment method")

# ===== Decorator (структурный паттерн) =====
class Order:
    def cost(self) -> float:
        return 100.0

class OrderDecorator(Order):
    def __init__(self, order: Order):
        self._order = order

    def cost(self) -> float:
        return self._order.cost()

class GiftWrapDecorator(OrderDecorator):
    def cost(self) -> float:
        return self._order.cost() + 10.0

# ===== Контекст =====
class CheckoutService:
    def __init__(self, payment_strategy: PaymentStrategy):
```

```
        self.payment_strategy = payment_strategy

    def checkout(self, order: Order) -> str:
        return self.payment_strategy.pay(order.cost())
```

Листинг test_shop_unittest.py

```
import unittest
from unittest.mock import Mock
from shop import (
    PaymentFactory,
    GiftWrapDecorator,
    Order,
    CheckoutService
)

class TestShop(unittest.TestCase):

    def test_factory_creates_card_payment(self):
        payment = PaymentFactory.create_payment("card")
        self.assertEqual(payment.pay(100), "Paid 100 by card")

    def test_decorator_adds_cost(self):
        order = GiftWrapDecorator(Order())
        self.assertEqual(order.cost(), 110.0)

    def test_checkout_with_mock_payment(self):
        mock_payment = Mock()
        mock_payment.pay.return_value = "Mock payment success"

        service = CheckoutService(mock_payment)
        result = service.checkout(Order())

        mock_payment.pay.assert_called_once()
        self.assertEqual(result, "Mock payment success")

if __name__ == "__main__":
    unittest.main()
```

Листинг checkout.feature

Feature: Checkout process

Scenario: Successful card payment

Given an order with gift wrap

When I pay by card

Then the payment result should be successful

Листинг checkout_steps.py

```
from behave import given, when, then
from shop import (
    Order,
    GiftWrapDecorator,
    PaymentFactory,
    CheckoutService
)

@given("an order with gift wrap")
def step_order(context):
    context.order = GiftWrapDecorator(Order())

@when("I pay by card")
def step_pay(context):
    payment = PaymentFactory.create_payment("card")
    service = CheckoutService(payment)
    context.result = service.checkout(context.order)

@then("the payment result should be successful")
def step_result(context):
    assert "Paid" in context.result
```

Результат выполнения программы:

```
koroligor@MacBook-Pro-Igor lab4 % python3 -m behave
USING RUNNER: behave.runner:Runner
Feature: Checkout process # features/checkout.feature:1

  Scenario: Successful card payment # features/checkout.feature:3
    Given an order with gift wrap # features/steps/checkout_steps.py:9 0.000s
    When I pay by card # features/steps/checkout_steps.py:13 0.000s
    Then the payment result should be successful # features/steps/checkout_steps.py:19 0.000s

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 0.000s
koroligor@MacBook-Pro-Igor lab4 % python3 -m unittest discover
...
-----
Ran 3 tests in 0.000s
OK
koroligor@MacBook-Pro-Igor lab4 %
```